

Riferimenti

Riferimenti

Come usare i riferimenti in C++ per creare alias alle variabili.

Cos'è un riferimento?

Supponi di chiamarti "Marco" ma tutti gli amici ti chiamano "Max". Sei sempre la stessa persona — solo con due nomi diversi.

Un **riferimento** in C++ funziona così: è un secondo nome per la stessa variabile. Se modifichi il valore tramite un nome, l'altro nome vede subito la modifica — perché si riferiscono alla stessa cosa.

I riferimenti sono più semplici e sicuri dei puntatori, e si usano moltissimo in C++.

Come creare un riferimento

Si usa `&` dopo il tipo nella dichiarazione:

```
int x = 42;
int& ref = x;    // ref è un altro nome per x

cout << x << endl;    // 42
cout << ref << endl;  // 42 (stessa variabile, stesso valore)

ref = 100;        // modifica il valore tramite ref
cout << x << endl;    // 100 – anche x è cambiata!
```

`ref` e `x` puntano alla stessa casella in memoria. Cambiare uno cambia l'altro.

Differenze tra riferimenti e puntatori

Caratteristica	Riferimento (&)	Puntatore (*)
Come si usa	<code>ref</code> (direttamente)	<code>*ptr</code> (con la stella)
Può essere nullo?	No	Sì (<code>nullptr</code>)
Può cambiare a cosa si riferisce?	No	Sì
Deve essere inizializzato subito?	Sì	No (ma è pericoloso)

I riferimenti sono più comodi e sicuri: una volta creati, puntano sempre alla stessa variabile.

Il caso d'uso principale: funzioni che modificano variabili

Normalmente, quando passi una variabile a una funzione, la funzione ne riceve una **copia**. Le modifiche alla copia non cambiano l'originale:

```
void raddoppiaValore(int x) {  
    x = x * 2;    // modifica solo la copia  
}  
  
int main() {  
    int numero = 5;  
    raddoppiaValore(numero);  
    cout << numero << endl;    // 5 - non è cambiato!  
}
```

Con un riferimento, la funzione lavora direttamente sull'originale:

```

void raddoppiaRiferimento(int& x) {
    x = x * 2;    // modifica la variabile originale
}

int main() {
    int numero = 5;
    raddoppiaRiferimento(numero);    // nessun &, si passa direttamente
    cout << numero << endl;        // 10 – è cambiato!
}

```

Il `&` nella **dichiarazione del parametro** dice: "non fare una copia, lavora sull'originale".

Scambiare due variabili

Con i riferimenti, il classico scambio di due variabili diventa molto leggibile:

```

void scambia(int& a, int& b) {
    int temp = a;    // salva a
    a = b;           // a prende il valore di b
    b = temp;        // b prende il vecchio valore di a
}

int main() {
    int x = 10, y = 20;
    cout << x << " " << y << endl;    // 10 20

    scambia(x, y);

    cout << x << " " << y << endl;    // 20 10
    return 0;
}

```

Riferimento costante: leggere senza copiare

Se passi una stringa o un oggetto grande a una funzione, il computer ne fa una copia — spreco di tempo e memoria.

Con `const &` passi il riferimento (nessuna copia) ma impedischi le modifiche:

```

// Senza const ref: copia l'intera stringa (lento per stringhe lunghe)
void stampa1(string s) {
    cout << s << endl;
}

// Con const ref: nessuna copia, non si può modificare (veloce e sicuro)
void stampa2(const string& s) {
    cout << s << endl;
    // s = "altro"; // ERRORE: è const, non si può modificare
}

int main() {
    string testo = "Una stringa molto lunga...";
    stampa1(testo); // fa una copia
    stampa2(testo); // non fa nessuna copia – più efficiente!
}

```

La regola pratica: usa `const string&` (e in generale `const Tipo&`) per passare oggetti grandi in sola lettura.

Riferimenti nel ciclo for

I riferimenti sono utili anche nei cicli `for` quando vuoi modificare gli elementi di una sequenza:

```

int numeri[] = {1, 2, 3, 4, 5};

// Senza riferimento: modifica solo una copia, l'array non cambia
for (int n : numeri) {
    n *= 2; // cambia la copia locale, non l'array
}

// Con riferimento: modifica l'array originale
for (int& n : numeri) {
    n *= 2; // modifica l'elemento vero dell'array
}

for (int n : numeri) {
    cout << n << " "; // 2 4 6 8 10
}

```

Esempio pratico completo

```
#include <iostream>
#include <string>
using namespace std;

// Riceve nome per riferimento costante: nessuna copia, non modificabile
void stampaInfo(const string& nome, int eta) {
    cout << "Nome: " << nome << ", Eta: " << eta << endl;
}

// Modifica il punteggio direttamente
void aggiungiPunti(int& punteggio, int bonus) {
    punteggio += bonus;
}

int main() {
    string giocatore = "Alice";
    int punti = 100;

    stampaInfo(giocatore, 16); // passa il riferimento costante

    aggiungiPunti(punti, 50); // punti viene modificato direttamente
    cout << "Punti aggiornati: " << punti << endl; // 150

    return 0;
}
```

Output:

```
Nome: Alice, Eta: 16
Punti aggiornati: 150
```